

CampOps Implementation Guide

Practical Migration to Modular Monolith

Version: 1.0

Last Updated: November 4, 2025

Stack: Next.js 15, Express.js, Supabase, TypeScript

Repository: <https://github.com/synaptic-shift-ai/Camp-OS>

Table of Contents

1. [Getting Started](#)
2. [Step-by-Step Migration](#)
3. [Code Examples](#)
4. [Testing Strategy](#)
5. [Deployment Guide](#)

Getting Started

Prerequisites

Your current setup already has:

- ✓ Next.js 15 with App Router
- ✓ Express.js API server
- ✓ Supabase (PostgreSQL + RLS)
- ✓ Stripe integration
- ✓ Comprehensive test suite (Vitest, Playwright)
- ✓ CI/CD with GitHub Actions

What We're Adding

New folder structure:

```
src/
├── shared/                # NEW - Shared kernel
│   ├── domain/
│   ├── infrastructure/
│   └── types/
├── modules/              # NEW - Business modules
│   ├── SiteManagement/
│   └── PropertyManagement/
```

```

|   |— GuestManagement/
|   |— BookingEngine/
|   |— Financial/
|— app/                                # EXISTING - Next.js pages
|— components/                        # EXISTING - React components
|— lib/                               # EXISTING - Utilities

```

Step-by-Step Migration

Week 1: Shared Kernel Setup

Step 1.1: Create Shared Domain Classes

Create: `src/shared/domain/Entity.ts`

```

export abstract class Entity<T> {
  protected readonly _id: T;
  protected readonly _createdAt: Date;
  protected _updatedAt: Date;

  constructor(id: T, createdAt?: Date) {
    this._id = id;
    this._createdAt = createdAt || new Date();
    this._updatedAt = new Date();
  }

  get id(): T {
    return this._id;
  }

  get createdAt(): Date {
    return this._createdAt;
  }

  get updatedAt(): Date {
    return this._updatedAt;
  }

  protected touch(): void {
    this._updatedAt = new Date();
  }

  equals(entity: Entity<T>): boolean {
    if (!(entity instanceof this.constructor)) {
      return false;
    }
    return this._id === entity._id;
  }
}

```

Create: `src/shared/domain/AggregateRoot.ts`

```

import { Entity } from './Entity';
import { DomainEvent } from './DomainEvent';

export abstract class AggregateRoot<T> extends Entity<T> {
  private _domainEvents: DomainEvent[] = [];

  get domainEvents(): ReadonlyArray<DomainEvent> {
    return this._domainEvents;
  }

  protected addDomainEvent(event: DomainEvent): void {
    this._domainEvents.push(event);
  }

  clearDomainEvents(): void {
    this._domainEvents = [];
  }

  protected markAsModified(): void {
    this.touch();
  }
}

```

Create: src/shared/domain/ValueObject.ts

```

export abstract class ValueObject {
  abstract equals(other: ValueObject): boolean;

  protected shallowEquals(other: any): boolean {
    if (!other || typeof other !== 'object') {
      return false;
    }

    const keys = Object.keys(this);
    return keys.every(key => this[key] === other[key]);
  }
}

```

Create: src/shared/domain/DomainEvent.ts

```

import { v4 as uuidv4 } from 'uuid';

export abstract class DomainEvent {
  public readonly eventId: string;
  public readonly occurredAt: Date;
  public readonly companyId: string;
  public readonly eventType: string;

  constructor(companyId: string) {
    this.eventId = uuidv4();
    this.occurredAt = new Date();
    this.companyId = companyId;
    this.eventType = this.constructor.name;
  }
}

```

```

    }

    abstract toJSON(): Record<string, any>;
  }

```

Step 1.2: Create Event Bus

Create: src/shared/infrastructure/eventBus/IEventBus.ts

```

import { DomainEvent } from '../../domain/DomainEvent';

export type EventHandler<T> extends DomainEvent = DomainEvent & {
  handle(event: T): Promise<void>;
};

export interface IEventBus {
  publish(event: DomainEvent): Promise<void>;
  subscribe<T> extends DomainEvent & {
    eventType: new (...args: any[]) => T,
    handler: EventHandler<T>;
  }(): void;
}

```

Create: src/shared/infrastructure/eventBus/InMemoryEventBus.ts

```

import { IEventBus, EventHandler } from './IEventBus';
import { DomainEvent } from '../../domain/DomainEvent';

export class InMemoryEventBus implements IEventBus {
  private handlers = new Map<string, EventHandler[]>();
  private eventStore: DomainEvent[] = [];

  subscribe<T> extends DomainEvent & {
    eventType: new (...args: any[]) => T,
    handler: EventHandler<T>;
  }(): void {
    const eventName = eventType.name;

    if (!this.handlers.has(eventName)) {
      this.handlers.set(eventName, []);
    }

    this.handlers.get(eventName)!.push(handler as EventHandler);
    console.log(`✔ Subscribed handler to ${eventName}`);
  }

  async publish(event: DomainEvent): Promise<void> {
    const eventName = event.constructor.name;

    // Store for audit
    this.eventStore.push(event);

    const handlers = this.handlers.get(eventName) || [];
  }
}

```

```

    if (handlers.length === 0) {
      console.warn(`⚠ No handlers registered for ${eventName}`);
      return;
    }

    console.log(`📡 Publishing ${eventName} to ${handlers.length} handler(s)`);

    const results = await Promise.allSettled(
      handlers.map(handler => handler.handle(event))
    );

    results.forEach((result, index) => {
      if (result.status === 'rejected') {
        console.error(
          `❌ Error in handler ${index} for ${eventName}:`,
          result.reason
        );
      }
    });
  }

  getEventStore(): ReadonlyArray<DomainEvent> {
    return this.eventStore;
  }
}

```

Step 1.3: Create Supabase Context Wrapper

Create: src/shared/infrastructure/database/SupabaseContext.ts

```

import { createClient, SupabaseClient } from '@supabase/supabase-js';
import type { Database } from '@database/types';

export class SupabaseContext {
  private client: SupabaseClient<Database>;

  constructor(supabaseUrl: string, supabaseKey: string) {
    this.client = createClient<Database>(supabaseUrl, supabaseKey);
  }

  getClient(): SupabaseClient<Database> {
    return this.client;
  }

  // Helper for setting company context (RLS)
  async setCompanyContext(companyId: string): Promise<void> {
    // Supabase RLS policies will use JWT claims
    // This is handled automatically by your auth middleware
  }

  // Transaction helper
  async transaction<T>(<
    callback: (client: SupabaseClient<Database>) => Promise<T>
  ): Promise<T> {

```

```

    // Supabase doesn't expose explicit transactions in the same way
    // We'll handle this at the application level
    return callback(this.client);
  }
}

```

Week 2: First Module - Site Management

Step 2.1: Create Domain Layer

Create: src/modules/SiteManagement/domain/valueObjects/Pricing.ts

```

import { ValueObject } from '@shared/domain/ValueObject';

export class Pricing extends ValueObject {
  constructor(
    public readonly basePrice: number,
    public readonly weekendPrice: number | null = null,
    public readonly seasonalPricing: SeasonalPrice[] = []
  ) {
    super();
    this.validate();
  }

  private validate(): void {
    if (this.basePrice < 0) {
      throw new Error('Base price cannot be negative');
    }
    if (this.weekendPrice !== null && this.weekendPrice < 0) {
      throw new Error('Weekend price cannot be negative');
    }
  }

  getPriceForDate(date: Date): number {
    // Check seasonal pricing
    for (const seasonal of this.seasonalPricing) {
      if (date >= seasonal.startDate && date <= seasonal.endDate) {
        return seasonal.price;
      }
    }

    // Check if weekend
    const dayOfWeek = date.getDay();
    if (this.weekendPrice && (dayOfWeek === 5 || dayOfWeek === 6)) {
      return this.weekendPrice;
    }

    return this.basePrice;
  }

  equals(other: Pricing): boolean {
    if (!(other instanceof Pricing)) return false;

    return (

```

```

        this.basePrice === other.basePrice &&&
        this.weekendPrice === other.weekendPrice &&&
        JSON.stringify(this.seasonalPricing) === JSON.stringify(other.seasonalPricing)
    );
}

toJSON() {
    return {
        basePrice: this.basePrice,
        weekendPrice: this.weekendPrice,
        seasonalPricing: this.seasonalPricing
    };
}
}

export interface SeasonalPrice {
    season: string;
    startDate: Date;
    endDate: Date;
    price: number;
}

```

Create: src/modules/SiteManagement/domain/entities/Site.ts

```

import { AggregateRoot } from '@shared/domain/AggregateRoot';
import { Pricing } from '../valueObjects/Pricing';
import { SiteCreatedEvent } from '../events/SiteCreatedEvent';
import { SiteStatusChangedEvent } from '../events/SiteStatusChangedEvent';

export enum SiteType {
    TENT = 'tent',
    RV = 'rv',
    CABIN = 'cabin',
    GLAMPING = 'glamping',
    YURT = 'yurt',
    OTHER = 'other'
}

export enum SiteStatus {
    AVAILABLE = 'available',
    RESERVED = 'reserved',
    BOOKED = 'booked',
    OCCUPIED = 'occupied',
    HOUSEKEEPING = 'housekeeping',
    MAINTENANCE = 'maintenance',
    UNAVAILABLE = 'unavailable'
}

interface Hookup {
    type: string;
    amperage?: number;
    pressure?: string;
    available: boolean;
}

```

```

export class Site extends AggregateRoot<string> {
  private constructor(
    id: string,
    private _companyId: string,
    private _propertyId: string,
    private _siteNumber: string,
    private _siteName: string | null,
    private _siteType: SiteType,
    private _maxOccupancy: number,
    private _maxVehicles: number,
    private _sizeSqft: number,
    private _hookups: Hookup[],
    private _amenities: string[],
    private _status: SiteStatus,
    private _pricing: Pricing,
    private _allowPets: boolean,
    private _petFee: number,
    private _adaAccessible: boolean,
    createdAt?: Date
  ) {
    super(id, createdAt);
  }

  static create(params: CreateSiteParams): Site {
    const id = crypto.randomUUID();

    // Validation
    if (!params.siteNumber || params.siteNumber.trim().length === 0) {
      throw new Error('Site number is required');
    }
    if (params.maxOccupancy <= 0) {
      throw new Error('Max occupancy must be greater than 0');
    }

    const site = new Site(
      id,
      params.companyId,
      params.propertyId,
      params.siteNumber,
      params.siteName || null,
      params.siteType,
      params.maxOccupancy,
      params.maxVehicles,
      params.sizeSqft,
      params.hookups,
      params.amenities,
      SiteStatus.AVAILABLE,
      params.pricing,
      params.allowPets,
      params.petFee || 0,
      params.adaAccessible
    );

    // Raise domain event
    site.addDomainEvent(
      new SiteCreatedEvent(

```



```

        params.companyId,
        id,
        params.propertyId,
        params.siteNumber,
        params.siteType
    )
);

return site;
}

static reconstitute(params: ReconstituteSiteParams): Site {
    return new Site(
        params.id,
        params.companyId,
        params.propertyId,
        params.siteNumber,
        params.siteName,
        params.siteType,
        params.maxOccupancy,
        params.maxVehicles,
        params.sizeSqft,
        params.hookups,
        params.amenities,
        params.status,
        params.pricing,
        params.allowPets,
        params.petFee,
        params.adaAccessible,
        params.createdAt
    );
}

// Business logic methods
markAsReserved(): void {
    if (this._status === SiteStatus.MAINTENANCE) {
        throw new Error('Cannot reserve site under maintenance');
    }
    if (this._status === SiteStatus.UNAVAILABLE) {
        throw new Error('Site is unavailable');
    }
}

const oldStatus = this._status;
this._status = SiteStatus.RESERVED;
this.markAsModified();

this.addDomainEvent(
    new SiteStatusChangedEvent(
        this._companyId,
        this.id,
        oldStatus,
        this._status
    )
);
}

```

```

markAsAvailable(): void {
  this._status = SiteStatus.AVAILABLE;
  this.markAsModified();
}

updatePricing(newPricing: Pricing): void {
  this._pricing = newPricing;
  this.markAsModified();
}

// Getters
get companyId(): string { return this._companyId; }
get propertyId(): string { return this._propertyId; }
get siteNumber(): string { return this._siteNumber; }
get siteName(): string | null { return this._siteName; }
get siteType(): SiteType { return this._siteType; }
get maxOccupancy(): number { return this._maxOccupancy; }
get status(): SiteStatus { return this._status; }
get pricing(): Pricing { return this._pricing; }

toJSON() {
  return {
    id: this.id,
    companyId: this._companyId,
    propertyId: this._propertyId,
    siteNumber: this._siteNumber,
    siteName: this._siteName,
    siteType: this._siteType,
    maxOccupancy: this._maxOccupancy,
    maxVehicles: this._maxVehicles,
    sizeSqft: this._sizeSqft,
    hookups: this._hookups,
    amenities: this._amenities,
    status: this._status,
    pricing: this._pricing.toJSON(),
    allowPets: this._allowPets,
    petFee: this._petFee,
    adaAccessible: this._adaAccessible,
    createdAt: this.createdAt,
    updatedAt: this.updatedAt
  };
}
}

export interface CreateSiteParams {
  companyId: string;
  propertyId: string;
  siteNumber: string;
  siteName?: string;
  siteType: SiteType;
  maxOccupancy: number;
  maxVehicles: number;
  sizeSqft: number;
  hookups: Hookup[];
  amenities: string[];
  pricing: Pricing;
}

```

```

    allowPets: boolean;
    petFee?: number;
    adaAccessible: boolean;
  }

export interface ReconstituteSiteParams extends CreateSiteParams {
  id: string;
  status: SiteStatus;
  createdAt: Date;
}

```

Create: src/modules/SiteManagement/domain/events/SiteCreatedEvent.ts

```

import { DomainEvent } from '@shared/domain/DomainEvent';
import { SiteType } from '../entities/Site';

export class SiteCreatedEvent extends DomainEvent {
  constructor(
    companyId: string,
    public readonly siteId: string,
    public readonly propertyId: string,
    public readonly siteNumber: string,
    public readonly siteType: SiteType
  ) {
    super(companyId);
  }

  toJSON() {
    return {
      eventId: this.eventId,
      eventType: this.eventType,
      occurredAt: this.occurredAt,
      companyId: this.companyId,
      data: {
        siteId: this.siteId,
        propertyId: this.propertyId,
        siteNumber: this.siteNumber,
        siteType: this.siteType
      }
    };
  }
}

```

Step 2.2: Create Application Layer

Create: src/modules/SiteManagement/application/commands/CreateSite/CreateSiteCommand.ts

```

export class CreateSiteCommand {
  constructor(
    public readonly companyId: string,
    public readonly propertyId: string,
    public readonly siteNumber: string,
    public readonly siteType: string,

```

```

    public readonly maxOccupancy: number,
    public readonly maxVehicles: number,
    public readonly sizeSqft: number,
    public readonly basePrice: number,
    public readonly weekendPrice: number | null,
    public readonly hookups: any[],
    public readonly amenities: string[],
    public readonly allowPets: boolean,
    public readonly petFee: number,
    public readonly adaAccessible: boolean
  ) {}
}

```

Create: src/modules/SiteManagement/application/commands/CreateSite/CreateSiteHandler.ts

```

import { CreateSiteCommand } from './CreateSiteCommand';
import { ISiteRepository } from '../../../domain/repositories/ISiteRepository';
import { IEventBus } from '@shared/infrastructure/eventBus/IEventBus';
import { Site, SiteType } from '../../../domain/entities/Site';
import { Pricing } from '../../../domain/valueObjects/Pricing';
import { SiteDTO } from './SiteDTO';

export class CreateSiteHandler {
  constructor(
    private readonly siteRepository: ISiteRepository,
    private readonly eventBus: IEventBus
  ) {}

  async execute(command: CreateSiteCommand): Promise<SiteDTO> {
    // Create pricing value object
    const pricing = new Pricing(
      command.basePrice,
      command.weekendPrice
    );

    // Create site aggregate
    const site = Site.create({
      companyId: command.companyId,
      propertyId: command.propertyId,
      siteNumber: command.siteNumber,
      siteType: command.siteType as SiteType,
      maxOccupancy: command.maxOccupancy,
      maxVehicles: command.maxVehicles,
      sizeSqft: command.sizeSqft,
      hookups: command.hookups,
      amenities: command.amenities,
      pricing,
      allowPets: command.allowPets,
      petFee: command.petFee,
      adaAccessible: command.adaAccessible
    });

    // Persist
    await this.siteRepository.save(site);
  }
}

```

```

    // Publish domain events
    for (const event of site.domainEvents) {
      await this.eventBus.publish(event);
    }
    site.clearDomainEvents();

    // Return DTO
    return SiteDTO.fromEntity(site);
  }
}

```

Step 2.3: Create Infrastructure Layer

Create: src/modules/SiteManagement/infrastructure/persistence/SupabaseSiteRepository.ts

```

import { SupabaseClient } from '@supabase/supabase-js';
import { ISiteRepository } from '../../domain/repositories/ISiteRepository';
import { Site, SiteType, SiteStatus } from '../../domain/entities/Site';
import { Pricing } from '../../domain/valueObjects/Pricing';
import type { Database } from '@database/types';

export class SupabaseSiteRepository implements ISiteRepository {
  constructor(private supabase: SupabaseClient<Database>) {}

  async save(site: Site): Promise<void> {
    const { error } = await this.supabase
      .from('sites')
      .insert({
        id: site.id,
        property_id: site.propertyId,
        site_number: site.siteNumber,
        site_name: site.siteName,
        site_type: site.siteType,
        max_occupancy: site.maxOccupancy,
        base_price: site.pricing.basePrice,
        weekend_price_cents: site.pricing.weekendPrice,
        status: site.status,
        allow_pets: site.toJSON().allowPets,
        pet_fee: site.toJSON().petFee,
        ada_accessible: site.toJSON().adaAccessible,
        created_at: site.createdAt.toISOString(),
        updated_at: site.updatedAt.toISOString()
      });

    if (error) {
      throw new Error(`Failed to save site: ${error.message}`);
    }
  }

  async findById(id: string): Promise<Site | null> {
    const { data, error } = await this.supabase
      .from('sites')
      .select('*')
      .eq('id', id)
      .single();
  }
}

```

```

    if (error || !data) return null;

    return this.mapToEntity(data);
}

async findByPropertyId(propertyId: string): Promise<Site[]> {
    const { data, error } = await this.supabase
        .from('sites')
        .select('*')
        .eq('property_id', propertyId)
        .order('site_number');

    if (error) {
        throw new Error(`Failed to fetch sites: ${error.message}`);
    }

    return data.map(row => this.mapToEntity(row));
}

private mapToEntity(row: any): Site {
    const pricing = new Pricing(
        row.base_price,
        row.weekend_price_cents,
        row.seasonal_pricing || []
    );

    return Site.reconstitute({
        id: row.id,
        companyId: row.company_id,
        propertyId: row.property_id,
        siteNumber: row.site_number,
        siteName: row.site_name,
        siteType: row.site_type as SiteType,
        maxOccupancy: row.max_occupancy,
        maxVehicles: row.max_vehicles,
        sizeSqft: row.size_sqft,
        hookups: row.hookups || [],
        amenities: row.amenities || [],
        status: row.status as SiteStatus,
        pricing,
        allowPets: row.allow_pets,
        petFee: row.pet_fee || 0,
        adaAccessible: row.ada_accessible,
        createdAt: new Date(row.created_at)
    });
}
}

```

Step 2.4: Create API Layer

Create: `api/modules/SiteManagement/controllers/SiteController.ts`

```
import { Request, Response, NextFunction } from 'express';
import { CreateSiteHandler } from '@modules/SiteManagement/application/commands/CreateSi
import { CreateSiteCommand } from '@modules/SiteManagement/application/commands/CreateSi

export class SiteController {
  constructor(
    private readonly createSiteHandler: CreateSiteHandler
  ) {}

  async createSite(req: Request, res: Response, next: NextFunction) {
    try {
      const companyId = req.user.companyId; // From auth middleware
      const { propertyId } = req.params;

      const command = new CreateSiteCommand(
        companyId,
        propertyId,
        req.body.siteNumber,
        req.body.siteType,
        req.body.maxOccupancy,
        req.body.maxVehicles,
        req.body.sizeSqft,
        req.body.basePrice,
        req.body.weekendPrice,
        req.body.hookups || [],
        req.body.amenities || [],
        req.body.allowPets || false,
        req.body.petFee || 0,
        req.body.adaAccessible || false
      );

      const result = await this.createSiteHandler.execute(command);

      res.status(201).json({
        success: true,
        data: result,
        meta: {
          timestamp: new Date().toISOString(),
          requestId: req.id
        }
      });
    } catch (error) {
      next(error);
    }
  }
}
```

Create: `api/modules/SiteManagement/routes.ts`

```

import { Router } from 'express';
import { SiteController } from '../controllers/SiteController';
import { authMiddleware } from '@api/shared/middleware/auth';
import { companyMiddleware } from '@api/shared/middleware/company';

export function createSiteRoutes(controller: SiteController): Router {
  const router = Router();

  // Apply middleware
  router.use(authMiddleware());
  router.use(companyMiddleware());

  // Routes
  router.post('/properties/:propertyId/sites', (req, res, next) => {
    controller.createSite(req, res, next)
  });

  return router;
}

```

Step 2.5: Wire It All Together

Create: api/modules/SiteManagement/index.ts

```

import { SupabaseClient } from '@supabase/supabase-js';
import { IEventBus } from '@shared/infrastructure/eventBus/IEventBus';
import { SupabaseSiteRepository } from '@modules/SiteManagement/infrastructure/persistence/SupabaseSiteRepository';
import { CreateSiteHandler } from '@modules/SiteManagement/application/commands/CreateSiteHandler';
import { SiteController } from '../controllers/SiteController';
import { createSiteRoutes } from '../routes';
import { Router } from 'express';

export class SiteManagementModule {
  private controller: SiteController;
  private router: Router;

  constructor(
    supabase: SupabaseClient,
    eventBus: IEventBus
  ) {
    // Infrastructure
    const siteRepository = new SupabaseSiteRepository(supabase);

    // Application
    const createSiteHandler = new CreateSiteHandler(siteRepository, eventBus);

    // API
    this.controller = new SiteController(createSiteHandler);
    this.router = createSiteRoutes(this.controller);
  }

  getRouter(): Router {
    return this.router;
  }
}

```



```
}  
}
```

Update: api/index.ts

```
import express from 'express';  
import { createClient } from '@supabase/supabase-js';  
import { InMemoryEventBus } from '@shared/infrastructure/eventBus/InMemoryEventBus';  
import { SiteManagementModule } from '../modules/SiteManagement';  
  
const app = express();  
app.use(express.json());  
  
// Initialize dependencies  
const supabase = createClient(  
  process.env.SUPABASE_URL!,  
  process.env.SUPABASE_SERVICE_KEY!  
);  
const eventBus = new InMemoryEventBus();  
  
// Initialize modules  
const siteManagementModule = new SiteManagementModule(supabase, eventBus);  
  
// Register routes  
app.use('/v1', siteManagementModule.getRouter());  
  
const PORT = process.env.API_PORT || 3001;  
app.listen(PORT, () => {  
  console.log(`👉 API server running on port ${PORT}`);  
});
```

Testing Strategy

Unit Tests

Test: tests/unit/modules/SiteManagement/domain/entities/Site.test.ts

```
import { describe, it, expect } from 'vitest';  
import { Site, SiteType, SiteStatus } from '@modules/SiteManagement/domain/entities/Site';  
import { Pricing } from '@modules/SiteManagement/domain/valueObjects/Pricing';  
  
describe('Site Entity', () => {  
  it('should create a site with valid params', () => {  
    const pricing = new Pricing(6500, 7500);  
  
    const site = Site.create({  
      companyId: 'company-123',  
      propertyId: 'property-456',  
      siteNumber: 'A-101',  
      siteType: SiteType.RV,  
      maxOccupancy: 6,  
      pricing,  
    });  
    expect(site).toBeDefined();  
    expect(site.companyId).toBe('company-123');  
    expect(site.propertyId).toBe('property-456');  
    expect(site.siteNumber).toBe('A-101');  
    expect(site.siteType).toBe(SiteType.RV);  
    expect(site.maxOccupancy).toBe(6);  
    expect(site.pricing).toBeDefined();  
    expect(site.pricing.minPrice).toBe(6500);  
    expect(site.pricing.maxPrice).toBe(7500);  
  });  
});
```

```

        maxVehicles: 2,
        sizeSqft: 1200,
        hookups: [],
        amenities: [],
        pricing,
        allowPets: true,
        petFee: 1500,
        adaAccessible: false
    });

    expect(site.siteNumber).toBe('A-101');
    expect(site.status).toBe(SiteStatus.AVAILABLE);
    expect(site.domainEvents.length).toBe(1);
});

it('should throw error for invalid site number', () => {
    const pricing = new Pricing(6500);

    expect(() => {
        Site.create({
            companyId: 'company-123',
            propertyId: 'property-456',
            siteNumber: '',
            siteType: SiteType.RV,
            maxOccupancy: 6,
            maxVehicles: 2,
            sizeSqft: 1200,
            hookups: [],
            amenities: [],
            pricing,
            allowPets: false,
            petFee: 0,
            adaAccessible: false
        });
    }).toThrow('Site number is required');
});

it('should mark site as reserved', () => {
    const pricing = new Pricing(6500);
    const site = Site.create({
        companyId: 'company-123',
        propertyId: 'property-456',
        siteNumber: 'A-101',
        siteType: SiteType.RV,
        maxOccupancy: 6,
        maxVehicles: 2,
        sizeSqft: 1200,
        hookups: [],
        amenities: [],
        pricing,
        allowPets: false,
        petFee: 0,
        adaAccessible: false
    });

    site.markAsReserved();

```

```

    expect(site.status).toBe(SiteStatus.RESERVED);
    expect(site.domainEvents.length).toBe(2); // Created + StatusChanged
  });
});

```

Integration Tests

Test: tests/integration/modules/SiteManagement/CreateSite.test.ts

```

import { describe, it, expect, beforeEach } from 'vitest';
import { createClient } from '@supabase/supabase-js';
import { InMemoryEventBus } from '@shared/infrastructure/eventBus/InMemoryEventBus';
import { SupabaseSiteRepository } from '@modules/SiteManagement/infrastructure/persistence/SupabaseSiteRepository';
import { CreateSiteHandler } from '@modules/SiteManagement/application/commands/CreateSiteHandler';
import { CreateSiteCommand } from '@modules/SiteManagement/application/commands/CreateSiteCommand';

describe('CreateSite Integration Test', () => {
  let supabase: any;
  let eventBus: InMemoryEventBus;
  let handler: CreateSiteHandler;

  beforeEach(() => {
    supabase = createClient(
      process.env.SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_KEY!
    );
    eventBus = new InMemoryEventBus();
    const repository = new SupabaseSiteRepository(supabase);
    handler = new CreateSiteHandler(repository, eventBus);
  });

  it('should create site and publish event', async () => {
    const command = new CreateSiteCommand(
      'company-123',
      'property-456',
      'TEST-101',
      'rv',
      6,
      2,
      1200,
      6500,
      7500,
      [],
      [],
      true,
      1500,
      false
    );

    const result = await handler.execute(command);

    expect(result.siteNumber).toBe('TEST-101');
    expect(eventBus.getEventStore().length).toBe(1);
    expect(eventBus.getEventStore()[0].eventType).toBe('SiteCreatedEvent');
  });
});

```

```
// Cleanup
await supabase.from('sites').delete().eq('id', result.id);
});
});
```

Deployment Guide

Environment Variables

Update your `.env` file:

```
# Existing variables
SUPABASE_URL=...
SUPABASE_SERVICE_KEY=...
STRIPE_SECRET_KEY=...

# New variables for modular architecture
EVENT_STORE_ENABLED=true
MODULE_LICENSES_CHECK=true
```

Build Configuration

Update `package.json`:

```
{
  "scripts": {
    "dev": "next dev",
    "api:dev": "tsx watch api/index.ts",
    "build": "next build && tsc -p tsconfig.api.json",
    "start": "next start",
    "api:start": "node dist/api/index.js",
    "test": "vitest",
    "test:integration": "vitest --config vitest.integration.config.ts"
  }
}
```

Docker Setup (Optional)

```
# Dockerfile
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --production

COPY . .
RUN npm run build
```

```
EXPOSE 3000 3001
```

```
CMD ["npm", "run", "start"]
```

Next Steps

1. **Complete Site Management Module** (Week 3-4)

- Add update, delete operations
- Implement availability checking
- Write comprehensive tests

2. **Extract Property Management** (Week 5-6)

- Follow same pattern as Site Management
- Wire to existing Supabase tables

3. **Continue with remaining modules** (Week 7+)

- Guest Management
- Booking Engine
- Financial Module

You're ready to start! Begin with the Shared Kernel setup and Site Management module. Each module follows this exact same pattern.